

Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea: Matematică și Informatică
Universitatea din București

Raport de Securitate (Mini Pentest)

Proiect 2: Break the Login

Coordonator:

Conf. Univ. Dr. Marius Iulian Mihăilescu

Student:

Lungu Bogdan-Cosmin CTI 461

1. Introducere

Videoclip: <https://youtu.be/XNIhV-ieDzA>

Github Repository: <https://github.com/Bogdan-Lungu/Break-the-login>

Descriere aplicatie si arhitectura:

Proiectul "AuthX" este o aplicatie web interna folosita de angajati pentru a accesa resurse sensibile (Dashboard). Scopul aplicatiei este de a demonstra in prima instantă un sistem de autentificare functional, dar extrem de vulnerabil (V1), iar ulterior dezvoltarea si securizarea acestuia la standarde moderne (V2).

Arhitectura se bazeaza pe un model Client-Server, unde backend-ul ofera endpoint-uri pentru autentificare, inregistrare, gestionare de sesiuni si resetare parole. Interfata grafica (UI) este simplificata folosind HTML/CSS, intreaga logica de validare si securitate aflandu-se in backend. Conform clarificarilor legate de "Implementare MVP", focusul principal este exclusiv pe functionalitatile de Auth (Authentication), componenta de CRUD/Search pentru tichete fiind ignorata (dupa intrebarea pusa de colegul meu pe canalul de teams), considerand Dashboard-ul ca resursa protejata.

2. Setup Mediu

Instalare VM, DB, framework:

Masina Virtuala: Dezvoltarea si testarea s-au facut pe ultima versiune de Kali Linux (2026.1).

User Kali: bogdan (schimbat ulterior la lungubogdan461)

Baza de date: S-a utilizat PostgreSQL v18, administrat prin interfata online pgAdmin 4 (logat cu contul personal lungubogdan02@gmail.com care se poate vedea si in imagini). Scriptul db_init.py a creat tabelele users si password_resets.

Framework si Dependente: S-a folosit Python 3 izolat intr-un mediu virtual (venv). Librariile instalate sunt: Flask (pentru server web), Flask-Session (pentru

sesiuni pe server-side), psycopg2-binary (pentru driverul de PostgreSQL) si bcrypt (pentru criptarea moderna a parolelor).

Tool-uri de testare: S-a utilizat Burp Suite (prin browserul integrat in suita) pentru capturarea, analiza si manipularea request-urilor / response-urilor HTTP.

IDE: Visual Studio Code.

Explicația Schemei Bazei de Date și Securitatea din Design:

Tabelul users stochează informațiile esențiale ale utilizatorilor. Observați prezența coloanelor locked (Boolean) și failed_logins (Integer). Aceste câmpuri au fost proiectate "Secure by Design" încă de la faza de inițializare a bazei de date, permițând implementarea ulterioară a funcționalității de Rate Limiting și Account Lockout direct la nivelul bazei de date. Astfel, starea de blocare persistă chiar și la repornirea serverului, protejând eficient și pe termen lung împotriva atacurilor de tip Brute Force.

Tabelul password_resets stochează token-urile pentru funcția de uitare a parolei, având un foreign key (user_id) restricționat prin ON DELETE CASCADE și un câmp obligatoriu expires_at care este vital pentru securitatea funcționalității de resetare.

Rularea aplicatiilor se face ruland scripturile din terminal: python app_v1.py (Port 5001) si python app_v2.py (Port 5002).

3. Implementare MVP (Auth)

Aplicația implementează flow-ul complet de autentificare (Register, Login, Dashboard protejat, Logout, si Forgot/Reset Password). După cum s-a confirmat, MVP-ul se axează strict pe mecanismele corecte de autentificare, aplicând verificări în baza de date PostgreSQL și returnând erori sau confirmări utilizatorului.

4. Prezentare vulnerabilitati, PoC, Impact, Fix si Re-test

4.1 Password Policy Slab

Prezentare vulnerabilitate:

In versiunea V1, nu exista validare pentru complexitatea parolei, permitand parole triviale de o singura cifra sau litera.

Cod Vulnerabil (extras din app_v1.py):

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        # 4.1 vulnerabilitate: password policy slab
        password_hash = hashlib.md5(password.encode()).hexdigest()
        # ... inserare directa in baza de date
    ...
```

Demonstrare atac (PoC):

Am inregistrat un cont (email test) cu o parola de doar 3 caractere ("123") in browser.

The image displays a security audit report on the left and a web browser on the right. The report, titled 'Curs: Dezvoltarea Aplicatiilor Software Securizate', includes a table of vulnerabilities. The table has columns for 'Nr. Incident', 'Categorie', 'Vulnerabilitate (Initial)', 'Impact', 'Cauza', and 'Mecanism de atac'. The first entry, '4.1', is categorized as 'Fragilitate' and describes a 'Parola slabă' (Weak Password) vulnerability. The impact is 'Nu există validare la înregistrare' (No validation during registration). The cause is 'Falta de validare a complexității parolei' (Lack of password complexity validation). The mechanism of attack is 'Inserarea unei parole triviale' (Inserting a trivial password). The browser on the right shows the 'Register (V1)' page with a successful registration of a user with email 'test@bogdan.ro' and password '123'.

Nr. Incident	Categorie	Vulnerabilitate (Initial)	Impact	Cauza	Mecanism de atac
4.1	Fragilitate	Parola slabă	Nu există validare la înregistrare	Falta de validare a complexității parolei	Inserarea unei parole triviale
4.2	Erori	Erori de validare a parolei	Existența unor parole care nu respectă cerințele de securitate	Existența unor parole care nu respectă cerințele de securitate	Existența unor parole care nu respectă cerințele de securitate

Analiza impact:

Atacatorul poate rula atacuri de tip Brute Force si Credential Stuffing cu un succes masiv, ghicind parolele slabe instantaneu cu ajutorul listelor.

Implementare fix:

In app_v2.py , am adaugat o functie is_password_complex(password) apelata pe backend la inregistrare/resetare. Aceasta forteaza o lungime de minim 8 caractere, litere mari/mici, cifre si simboluri speciale.

Cod Securizat (extras din app_v2.py):

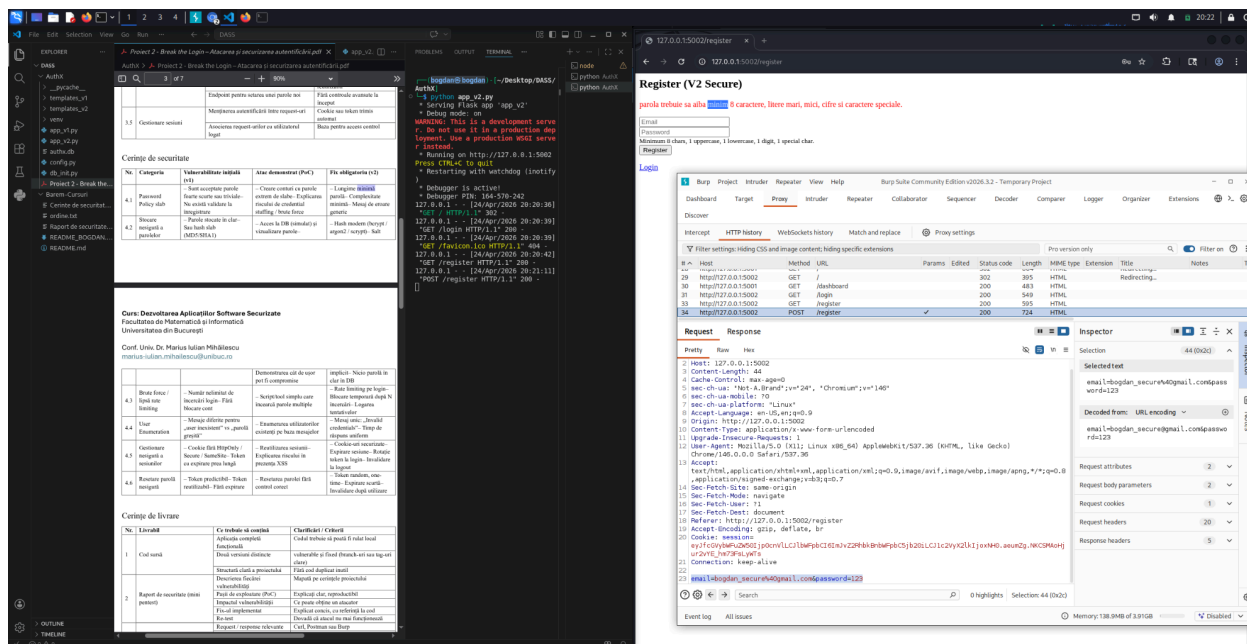
```
python
def is_password_complex(password):
    # 4.1 lungime minima parola si complexitate minima
    if len(password) < 8: return False
    if not any(char.isdigit() for char in password): return False
    if not any(char.isupper() for char in password): return False
    if not any(char.islower() for char in password): return False
    if not any(char in "!@#$%^&*()-_+[]{};:,.<>?" for char in password): return False
    return True

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        # 4.1 validare la inregistrare - respingere instante slabe
        if not is_password_complex(password):
            return render_template('register.html', error="parola trebuie sa aiba minim 8 caractere, litere mari, mici, cifre si caractere speciale.")
```

Re-test:

Inercarea de a inregistra o parola simpla pe V2 ("123") este refuzata automat cu mesaj explicativ.



4.2 Stocare nesigura a parolelor

Prezentare vulnerabilitate:

In V1, parolele sunt salvate folosind functia hashlib.md5() fara un salt criptografic. MD5 este considerat teoretic si practic "spart" in contextul stocarii de parole.

Cod Vulnerabil (extras din app_v1.py):

```
python
# 4.2 vulnerabilitate: stocare nesigura a parolelor - hash slab (md5) fara salt
password_hash = hashlib.md5(password.encode()).hexdigest()
```

Demonstrare atac (PoC):

Am extras datele din pgAdmin 4. Hash-ul vizibil pentru parola este extrem de scurt, comun tuturor instalatiilor (predictibil).

The screenshot displays a development environment with three main components:

- Left Panel (Document):** A document titled "Proiect 2 - Break the Login - Alina" contains a table of security checks and a table of security measures.
- Center Panel (Terminal):** A terminal window shows the execution of a Python script "app_v2.py". The output displays a password hash: "password_hash = b'1234567890'".
- Right Panel (pgAdmin 4):** The pgAdmin 4 interface shows a query result for the "password_hash" column. The result shows a common, predictable hash value for the password "1234567890".

Analiza impact:

In cazul in care baza de date este expusa (prin SQL Injection sau scurgeri de date), atacatorul poate folosi Rainbow Tables sau hashcat pentru a decipta offline milioane de parole in cateva secunde.

Implementare fix:

In V2, s-a importat biblioteca bcrypt . Functia bcrypt.hashpw genereaza un "salt" pseudo-randomizat pe 128-bit si implementeaza key stretching (algoritm costisitor ca timp), generand string-uri unice chiar si pentru aceeasi parola. Hashul incepe cu \$2b\$.

Cod Securizat (extras din app_v2.py):

```
python
# 4.2 hash modern (bcrypt) + salt implicit (nici o parola in clar in db)
password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
```

Re-test:

Dupa cum se vede in poza cu pgAdmin din V2 (imaginea de mai sus), noul hash este modern, de lungime fixa (60 caractere) si imun la Rainbow Tables.

4.3 Brute force / Lipsa Rate Limiting

Prezentare vulnerabilitate:

Endpoint-ul de /login din V1 nu monitorizeaza ritmul cererilor. Se pot trimite zeci de mii de parole intr-un timp extrem de scurt fara niciun mecanism de blocare.

Cod Vulnerabil (extras din app_v1.py):

```
python

# 4.3 vulnerabilitate: brute force / lipsa rate limiting - numar nelimitat de incercari login
password_hash = hashlib.md5(password.encode()).hexdigest()
if user['password_hash'] != password_hash:
    cur.close()
    conn.close()
    return "parola incorecta"
```

Demonstrare atac (PoC):

Folosind utilitarul Burp Suite -> Intruder (atac Sniper), am incarcat un "payload list" cu 10 parole gresite trimise catre emailul unui utilizator. Toate request-urile au intors "HTTP 200 OK", demonstrand ca atacul curge neintrerupt.

The screenshot displays the Burp Suite Intruder interface during a brute force attack. The 'Intruder' tab is active, showing a list of 10 payloads (passwords) being tested against the target URL 'http://127.0.0.1:5001'. The 'Results' table shows that all 10 attempts resulted in a 200 OK status, indicating successful logins. The 'Payloads' tab on the right shows the list of passwords used, including '123456', '123456789', '1234567890', '12345678901', '123456789012', '1234567890123', '12345678901234', '123456789012345', '1234567890123456', and '12345678901234567'. The 'Request' and 'Response' tabs show the details of the HTTP requests and responses, including the 'Content-Type' and 'User-Agent' headers.

No.	Companie	Vulnerabilitate (CVE)	Alte descrieri (PwC)	Fis obiective (PwC)
4.1	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...
4.2	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...

No.	Companie	Vulnerabilitate (CVE)	Alte descrieri (PwC)	Fis obiective (PwC)
4.1	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...
4.2	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...

No.	Companie	Vulnerabilitate (CVE)	Alte descrieri (PwC)	Fis obiective (PwC)
4.1	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...
4.2	Microsoft	MS16-016	... Crearea unui nou cont de utilizator...	... Crearea unui nou cont de utilizator...

Analiza impact:

Automatizarea ghicirii parolelor permite compromiterea conturilor care nu respecta politici foarte stricte de complexitate. Atacatorul nu este limitat decat de resursele serverului.

Implementare fix:

Pe V2, rate limiting-ul a fost aplicat direct in baza de date (Account Lockout).

Variabila failed_logins se incrementeaza la fiecare esec. Daca numarul de esecuri atinge valoarea 5, coloana locked devine

True , iar serverul opreste validările viitoare, returnand "cont blocat".

Cod Securizat (extras din app_v2.py):

```
python
# 4.3 rate limiting pe login - blocare temporara dupa n incercari. verificam daca e deja blocat
if user and user['locked']:
    cur.close()
    conn.close()
    return render_template('login.html', error="cont blocat. contactati administratorul.")

if not bcrypt.checkpw(password.encode('utf-8'), user['password_hash'].encode('utf-8')):
    # 4.3 logarea tentativelor - incrementam failed_logins si blocam dupa 5 incercari
    failed_logins = user['failed_logins'] + 1
    locked = True if failed_logins >= 5 else False
    cur.execute("UPDATE users SET failed_logins = %s, locked = %s WHERE id = %s", (failed_logins, locked, user
    ['id']))
    conn.commit()
```


4.4 User Enumeration

Prezentare vulnerabilitate:

Mesajele de eroare de la login difera: "utilizator inexistent" cand nu gaseste email-ul si "parola incorecta" cand il gaseste.

Cod Vulnerabil (extras din app_v1.py):

```
```python
 # 4.4 vulnerabilitate: user enumeration - mesaje diferite
 if not user:
 return "utilizator inexistent"

 password_hash = hashlib.md5(password.encode()).hexdigest()
 if user['password_hash'] != password_hash:
 return "parola incorecta"
```
```

Demonstrare atac (PoC):

Am trimis prin Burp Repeater doua request-uri si am primit raspunsurile diferite descrise mai sus.

The screenshot shows a development environment with a terminal, a file explorer, and a web browser. The terminal displays the output of a Burp Suite request, showing a 401 Unauthorized response. The file explorer shows a directory structure with files like app_v2.py and app_v2.py. The web browser shows the Burp Suite interface with a request and response view. The response is a 401 Unauthorized status with a 'utilizator inexistent' (user does not exist) message.

The screenshot shows a development environment with a terminal, a file explorer, and a web browser. The terminal displays the output of a Burp Suite request, showing a 401 Unauthorized response. The file explorer shows a directory structure with files like app_v2.py and app_v2.py. The web browser shows the Burp Suite interface with a request and response view. The response is a 401 Unauthorized status with a 'parola incorecta' (wrong password) message.

Analiza impact:

Atacatorul poate incarca liste uriase de adrese de email publice in Burp Intruder pentru a gasi ce utilizatori au conturi in aceasta aplicatie corporativa, construind liste pentru atacuri ulterioare.

Implementare fix:

Am unificat mesajul de raspuns in V2: "credentiale invalide". De asemenea, ca masura avansata de securitate impotriva "Timing Attacks" (enumerare prin timpul de raspuns), daca user-ul nu exista in DB, V2 inca calculeaza un hash dummy folosind bcrypt.checkpw inainte de a returna eroarea.

Cod Securizat (extras din app_v2.py):

```
```python
 # 4.4 anti user enumeration - mesaj unic: "credentiale invalide"
 generic_error = "credentiale invalide"

 if not user:
 # 4.4 timp de raspuns uniform - simulam hash-uirea pentru a preveni timing attacks
 bcrypt.checkpw(password.encode('utf-8'), bcrypt.gensalt())
 cur.close()
 conn.close()
 return render_template('login.html', error=generic_error)
```
```

Re-test:

Trimitand aceleasi payload-uri in V2, mesajul ramas pe ecran a fost identic pentru ambele situatii.

The screenshot displays a development environment with three main components:

- PDF Document:** A document titled "Certe de securitate" (Security Certificates) and "Certe de Securitate Aplicațiilor Software Securizate" (Security Certificates of Secured Software Applications). It contains two tables:

| No. | Categorie | Vulnerabilități inițiale (V1) | Alte documente (P&V) | Pia vulnerabilități (V2) |
|-----|------------------------|--|--|--|
| 4.1 | Permisul Policy date | - Sunt acceptate parole care conțin caractere speciale, inclusiv caracterele de control și caracterele de control de sfârșit de linie. | - Creați conținut cu parole care conțin caractere speciale, inclusiv caracterele de control și caracterele de control de sfârșit de linie. | - Lungime minimă: 8 caractere. Complexitate minimă: 1 caracter din fiecare categorie: alfanumeric, alfabetic, numeric, special și control. |
| 4.2 | Securitatea sesiunilor | - Fiecare sesiune de utilizator este validată în mod continuu. | - Asigurați o sesiune de utilizator în mod continuu. | - Fiecare sesiune de utilizator este validată în mod continuu. |

| No. | Descriere | Ce trebuie să conțină | Caracteristici / Criterii |
|-----|--------------------------------------|---|---|
| 1 | Cod sursă | - Codul sursă este disponibil în mod public. | - Codul sursă este disponibil în mod public. |
| 2 | Reprezentarea securității (versiuni) | - Reprezentarea securității este disponibilă în mod public. | - Reprezentarea securității este disponibilă în mod public. |
| 3 | Documentația | - Documentația este disponibilă în mod public. | - Documentația este disponibilă în mod public. |

- Terminal:** Shows a command prompt with a password prompt: "Login (V2 Secure) credentiale invalide".
- Web Browser:** Shows a login page with a "Login (V2 Secure)" button and a "credentiale invalide" message.

The screenshot displays a development environment with three main components:

- PDF Document:** A document titled "Certe de securitate" (Security Certificates) and "Certe de Securitate Aplicațiilor Software Securizate" (Security Certificates of Secured Software Applications). It contains two tables:

| No. | Categorie | Vulnerabilități inițiale (V1) | Alte documente (P&V) | Pia vulnerabilități (V2) |
|-----|------------------------|--|--|--|
| 4.1 | Permisul Policy date | - Sunt acceptate parole care conțin caractere speciale, inclusiv caracterele de control și caracterele de control de sfârșit de linie. | - Creați conținut cu parole care conțin caractere speciale, inclusiv caracterele de control și caracterele de control de sfârșit de linie. | - Lungime minimă: 8 caractere. Complexitate minimă: 1 caracter din fiecare categorie: alfanumeric, alfabetic, numeric, special și control. |
| 4.2 | Securitatea sesiunilor | - Fiecare sesiune de utilizator este validată în mod continuu. | - Asigurați o sesiune de utilizator în mod continuu. | - Fiecare sesiune de utilizator este validată în mod continuu. |

| No. | Descriere | Ce trebuie să conțină | Caracteristici / Criterii |
|-----|--------------------------------------|---|---|
| 1 | Cod sursă | - Codul sursă este disponibil în mod public. | - Codul sursă este disponibil în mod public. |
| 2 | Reprezentarea securității (versiuni) | - Reprezentarea securității este disponibilă în mod public. | - Reprezentarea securității este disponibilă în mod public. |
| 3 | Documentația | - Documentația este disponibilă în mod public. | - Documentația este disponibilă în mod public. |

- Terminal:** Shows a command prompt with a password prompt: "Login (V2 Secure) credentiale invalide".
- Web Browser:** Shows a login page with a "Login (V2 Secure)" button and a "credentiale invalide" message.

4.5 Gestionare nesigura a sesiunilor

Prezentare vulnerabilitate:

Sesiunile in V1 sunt vulnerabile pe doua fronturi: cookie-urile sunt expuse la interceptari si atacuri XSS prin lipsa flag-urilor de securitate (HttpOnly , Secure), iar sesiunile nu sunt invalidate pe server la logout, permitand refolosirea lor (Session Reuse).

Cod Vulnerabil (extras din app_v1.py):

```
```python
4.5 vulnerabilitate: gestionare nesigura a sesiunilor, lipsa flag-uri
app.config['SESSION_COOKIE_HTTPONLY'] = False
app.config['SESSION_COOKIE_SECURE'] = False

@app.route('/logout')
def logout():
 # 4.5 vulnerabilitate: reutilizarea sesiunii (session.clear() lipseste intentionat)
 resp = make_response(redirect(url_for('login')))
 resp.set_cookie('session', '', expires=0)
 return resp
```
```

Demonstrare atac (PoC):

Dupa login, header-ul Set-Cookie returnat de V1 este extrem de slab. Apoi, la "Logout", am copiat manual cookie-ul vechi si l-am introdus in Burp Repeater incercand sa accesez GET /dashboard . Serverul mi-a permis accesul (HTTP 200 OK) fara probleme.

The screenshot displays a development environment with three main windows:

- Course Document:** A PDF titled "Curs: Dezvoltarea Aplicațiilor Software Securizate" (Course: Development of Secure Software Applications) from the Faculty of Mathematics and Informatics, University of Bucharest. It contains a table of contents and a list of exercises.
- Terminal:** Shows the execution of a Python script named "app_v2.py" using the command "python app_v2.py". The output indicates that the application is running on http://127.0.0.1:5000.
- Web Browser:** Displays the "Dashboard (V1)" page. The address bar shows "http://127.0.0.1:5000/dashboard". The page content includes a welcome message and a "Logout" link. The Burp Suite interface is visible, showing the request and response details for the GET /dashboard endpoint.

The screenshot displays a development environment with three main windows:

- Course Document:** A PDF titled "Curs: Dezvoltarea Aplicațiilor Software Securizate" (Course: Development of Secure Software Applications) from the Faculty of Mathematics and Informatics, University of Bucharest. It contains a table of contents and a list of exercises.
- Terminal:** Shows the execution of a Python script named "app_v2.py" using the command "python app_v2.py". The output indicates that the application is running on http://127.0.0.1:5000.
- Web Browser:** Displays the "Login (V1)" page. The address bar shows "http://127.0.0.1:5000/login". The page content includes a login form and a "Logout" link. The Burp Suite interface is visible, showing the request and response details for the GET /login endpoint.

Analiza impact:

Oricine fura token-ul (prin Javascript / sniffing) poate mentine accesul neautorizat mult timp dupa ce victima s-a deconectat, avand in vedere ca timpul de expirare este infinit (permanent).

Implementare fix:

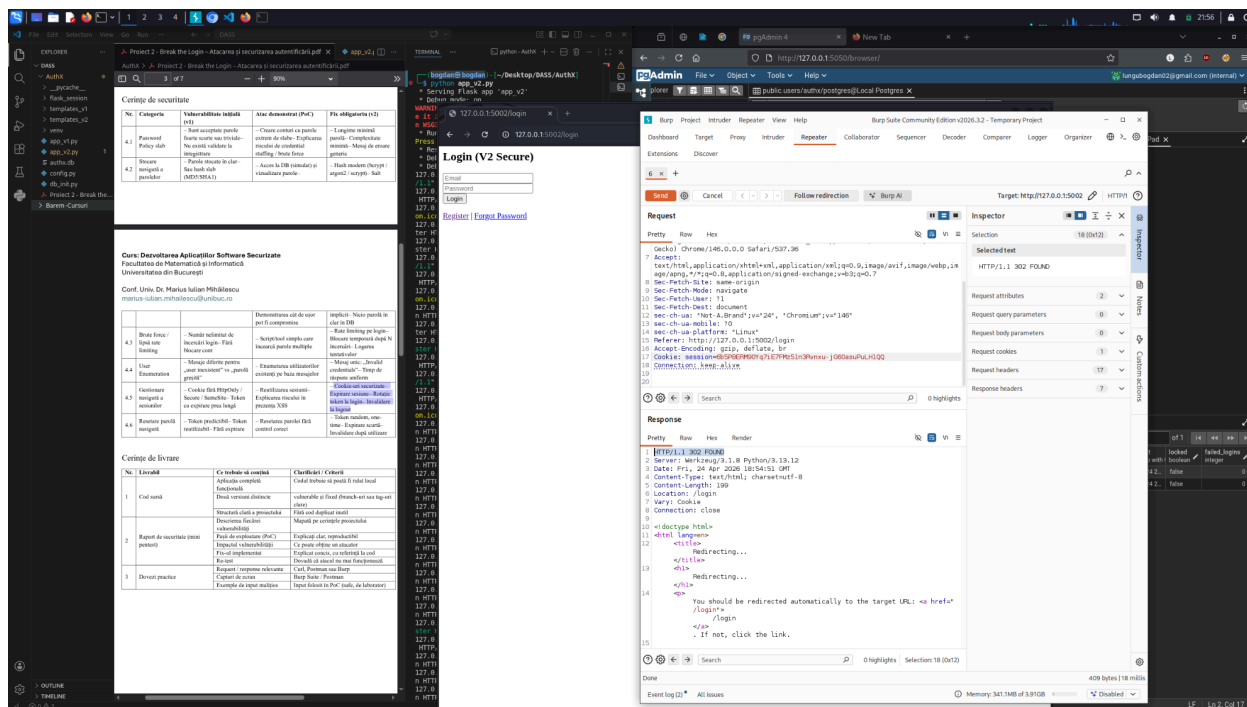
Pentru V2, m-am asigurat ca folosesc extensia Flask-Session pentru "Server-Side Sessions". Acum, cookie-ul de sesiune este generat cu flag-urile HttpOnly , Secure si SameSite=Lax . S-a implementat o expirare scurta (PERMANENT_SESSION_LIFETIME = 30 min). Cel mai important, la delogare (/logout), a fost inclus apelul session.clear() care sterge inregistrarea sesiunii din fisierul de server, anuland utilitatea cookie-ului local.

Cod Securizat (extras din app_v2.py):

```
```python
4.5 securizare: cookie-uri securizate (httponly, secure, samesite)
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SECURE'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(minutes=30)

@app.route('/logout')
def logout():
 # 4.5 invalidare la logout pe server
 session.clear()
 resp = make_response(redirect(url_for('login')))
 resp.set_cookie(app.config['SESSION_COOKIE_NAME'], '', expires=0, httponly=True, secure=True, samesite='Lax')
 return resp
```
```

Cookie-ul nou returnat are flag-urile necesare. Refolosind vechiul cookie in Burp Repeater dupa logout, serverul V2 refuza complet cererea cu o redirectionare HTTP/1.1 302 FOUND catre pagina de login.



4.6 Resetare parola nesigura

Prezentare vulnerabilitate:

Token-urile de resetare din V1 sunt predictibile si refolosibile la infinit.

Cod Vulnerabil (extras din app_v1.py):

```
python

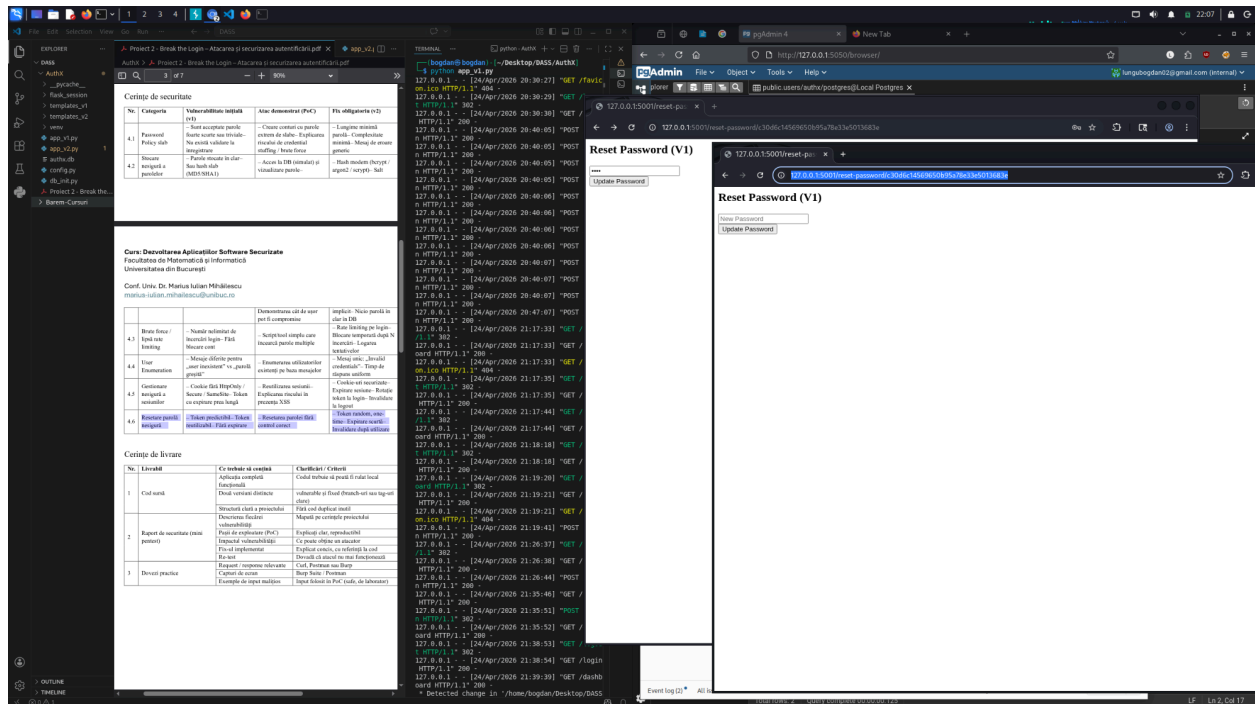
# 4.6 vulnerabilitate: resetare parola nesigura - token predictibil (doar un md5 al email-ului)
token = hashlib.md5(email.encode()).hexdigest()
# 4.6 vulnerabilitate: fara expirare la token (data din viitorul indepartat)
cur.execute("INSERT INTO password_resets (user_id, token, expires_at) VALUES (%s, %s, '9999-12-31')", (user
['id'], token))
```

Demonstrare atac (PoC):

Token-ul de resetare din bara de adresa s-a dovedit a fi valoarea de hash MD5 a adresei de email solicitate. Dupa ce am schimbat parola folosind link-ul, i-am dat Refresh paginii. Aplicatia mi-a permis sa rescriu o alta parola, deoarece nu a marcat acel link ca folosit.

The screenshot displays a web application security tool (Burp Suite) on the left and a web browser window on the right. The browser window shows a 'Reset Password' page with a link to reset the password. The HTTP history in Burp Suite shows a GET request to /reset-password with a token parameter. The token is a MD5 hash of the email address.

| No. | Urmare | Metoda | Adresa | Parametri | Stare | Tip | Comentarii |
|-----|--------|----------|--------------------------------------|--------------------------------------|-------|---------|------------|
| 1 | GET | HTTP/1.1 | http://127.0.0.1:5555/reset-password | token=9b08c146986086507bc13c5013043e | 200 | Success | |



Analiza impact:

Predictibilitatea asigura compromiterea prin "Account Takeover" la comanda hacker-ului. Faptul ca nu expira asigura o fereastra de vulnerabilitate permanenta.

Implementare fix:

In V2, token-ul a fost generat criptografic folosind biblioteca Python secrets (secrets.token_urlsafe). I-a fost impusa o expirare stricta de 15 minute, salvata in baza de date. Dupa completarea procesului de resetare, am adaugat query-ul de DELETE pentru a sterge inregistrarea token-ului (user_id = %s), impunand functionalitatea de "One-time use".

Cod Securizat (extras din app_v2.py):

```
python

# 4.6 token random, criptografic sigur
token = secrets.token_urlsafe(32)
# 4.6 expirare scurta (15 minute)
expires_at = datetime.now(timezone.utc) + timedelta(minutes=15)
cur.execute("INSERT INTO password_resets (user_id, token, expires_at) VALUES (%s, %s, %s)", (user['id'], token, expires_at))

# In functia reset_password - control one-time use
if request.method == 'POST':
    # ...
    # 4.6 one-time use - invalidare dupa utilizare
    cur.execute("DELETE FROM password_resets WHERE user_id = %s", (reset_entry['user_id'],))
```


Re-test:

Token-ul generat este random. Dupa o resetare reusita, incercarea de a accesa a doua oara acelasi URL s-a soldat cu mesajul "token invalid / expirat." generat din backend.

The screenshot displays a development environment with three main components:

- Left Panel (VS Code):** Shows a document titled "Proiect 2 - Break the Login - Alacarea si securizarea autentificarii.pdf". It contains two tables: "Cerinta de securitate" (Security Requirement) and "Cerinta de evaluare" (Evaluation Requirement).
- Terminal:** Displays a series of curl commands being executed against a local Postgres database. The commands include GET requests for /favicon.ico, /login, and /forgot-password, and a POST request for /login with a password. The output shows the database's response, including a 200 status code and a token.
- Browser:** Shows the login page of a web application. The URL is http://127.0.0.1:5000/login. The page has a title "Admin" and a form with fields for "Email" and "Password". The "Email" field is filled with "public.users/auth/postgres@local.postgres". The "Password" field is filled with "127.0.0.1:5000/forgot-password". The "Login" button is visible.

The screenshot displays a development environment with three main components:

- Left Panel (VS Code):** Shows a document titled "Proiect 2 - Break the Login - Alacarea si securizarea autentificarii.pdf". It contains two tables: "Cerinta de securitate" (Security Requirement) and "Cerinta de evaluare" (Evaluation Requirement).
- Terminal:** Displays a series of curl commands being executed against a local Postgres database. The commands include GET requests for /favicon.ico, /login, and /forgot-password, and a POST request for /login with a password. The output shows the database's response, including a 200 status code and a token.
- Browser:** Shows the login page of a web application. The URL is http://127.0.0.1:5000/login. The page has a title "Admin" and a form with fields for "Email" and "Password". The "Email" field is filled with "public.users/auth/postgres@local.postgres". The "Password" field is filled with "127.0.0.1:5000/forgot-password". The "Login" button is visible.

5. Audit & Logging

Conform clarificarilor primite pe canalul de comunicare (Teams) privind ignorarea componentelor de CRUD (tichete), am ales o abordare diferita pentru sistemul de logging. Astfel, in locul tabelului separat `audit_logs`, trasabilitatea evenimentelor de securitate, care este ceruta la sectiunea 'Audit & logging', din structura obligatorie a raportului, este asigurata prin:

1. Logging in Baza de Date: Campul `failed_logins` din tabelul `users` functioneaza ca un mecanism activ de auditare a tentativelor de login. Acesta permite monitorizarea comportamentului malitios si corelarea atacurilor de tip Brute Force direct la nivelul identitatii vizate.

2. Logging in Terminal (Flask): Serverul backend logheaza automat toate cererile HTTP (Request Logs), incluzand adresa IP, timestamp-ul si endpoint-ul accesat. Aceste loguri ofera vizibilitate completa asupra fluxului de date si a posibilelor scanari automate efectuate asupra aplicatiei. Aceasta structura a fost aleasa pentru a mentine scope-ul proiectului focusat pe mecanismele corecte de Authentication (Auth MVP), eliminand redundanta unui tabel separat de audit intr-un mediu de testare.

6. Concluzii

Acest proiect mi-a oferit perspectiva duala de Offensive si Defensive security asupra unuia dintre cele mai expuse module din web: Autentificarea.

Printre lectiile majore invatate se enumara faptul ca functiile utilitare implicite in framework-uri (precum `session normal` din Flask fara `Server-Side sessions` si fara flag-uri de `HttpOnly / Secure`) vin cu defecte considerabile out-of-the-box. Securitatea presupune "Defense in Depth" (Aparare in Adancime): blocarea atacului brut pe de-o parte (Rate Limiting), dar si generarea de timpi de raspuns constanti prin pre-calculari de BCrypt false (pentru a elimina scurgerile "tacute" din User Enumeration). Orice functionalitate expusa atacatorilor trebuie restrictionata si validata activ la nivel de server.